

Uber Original PYQ Style Practice

Product Based / Uber

Study Hub Premium Placement Pack

This is original study material for preparation. It is not a copied official or proprietary company paper.

Premium Study System

- Track: Uber.
- Pack type: PYQ / PYQ Practice.
- Target outcome: selection-ready confidence for SDE / Software Engineer.
- Core preparation areas: DSA, CS fundamentals, OOP/LLD, HLD basics, behavioral stories, resume projects.
- Use this pack like a mini-book: concept, table, example, practice, analysis, revision.
- Every topic should produce proof: solved questions, mock score, mistake note, interview answer, or resume evidence.

Output Quality Rules

- Read the concept in your own words, then solve without looking at notes.
- Convert every important idea into a comparison table, checklist, or one-line rule.
- Mark important traps as Common Mistake before the mock, not after losing marks.
- Keep a PYQ-style log: question pattern, topic, time taken, mistake reason, retest date.
- Use simple English; add Hindi/Hinglish meaning in brackets when it helps recall.

Priority Matrix

Area	Premium Standard	Proof Before Moving On
DSA	Pattern, invariant, dry run, complexity	2 solved problems per pattern without hints
CS Core	OS, DBMS, CN, OOP with examples	Explain each topic in 60 seconds
Project	Problem, architecture, your contribution, tradeoff	2-minute story plus deep-dive answers
Communication	Crisp intro, structured HR stories, calm tone	5 recorded answers reviewed once

Exam Tip

- Interviewers reward clean reasoning more than memorized code. Say brute force, improve it, dry run, then discuss complexity.

Common Mistake

- Jumping into code without constraints creates wrong edge cases. Ask size, duplicates, sorted/unsorted, and expected output first.

Original OA and Interview Practice

- This set is designed for Uber style product preparation: DSA depth, CS fundamentals, design thinking, project ownership, and behavioral clarity.

Section A: DSA Pattern Practice

- Q1. Return indices of two numbers adding to target. Explain hash map time and space complexity.
- Q2. Merge overlapping intervals and list the sorting invariant.
- Q3. Detect cycle in a linked list, then find the cycle start.
- Q4. Find shortest path in an unweighted graph and explain why BFS works.
- Q5. Find the longest substring without repeating characters. Explain sliding window movement.
- Q6. Search an element in a rotated sorted array. Explain the binary-search decision rule.

- Q7. Given daily temperatures, return days to wait for warmer temperature. Explain stack invariant.
- Q8. Count islands in a grid. Explain visited marking and boundary checks.

Section B: DP and Optimization

- Q9. Count ways to climb stairs with 1 or 2 steps, then optimize memory.
- Q10. Find maximum subarray sum and explain why local optimum works.
- Q11. Find coin change minimum coins. Define state, transition, base case, and impossible state.
- Q12. Given jobs with start, end, profit, select maximum profit non-overlapping jobs. Explain sorting + DP idea.

Section C: LLD and HLD

- Q13. LLD: Design a parking lot with classes, responsibilities, pricing, and edge cases.
- Q14. LLD: Design a rate limiter. Compare fixed window, sliding window, and token bucket.
- Q15. HLD: Design a tiny URL service. Cover APIs, storage, cache, collisions, and scaling.
- Q16. HLD: Design a notification system. Cover queues, retries, idempotency, and failure handling.

Section D: CS Core and Project Depth

- Q17. Explain database indexing and when an index can hurt performance.
- Q18. Explain process vs thread and how context switching affects performance.
- Q19. Explain HTTP status codes, cookies, JWT, and session tradeoffs.
- Q20. Describe a time you handled ambiguity while preparing for Uber. Use a real example.

Model Hints

- Q1 hint: hash map stores complement or seen values; time $O(n)$, space $O(n)$.
- Q3 hint: Floyd cycle detection gives meeting point; reset one pointer to head to find cycle start.
- Q8 hint: DFS/BFS from every unvisited land cell; each cell is visited once.
- Q14 hint: mention data model, request key, time bucket, concurrency, and cleanup.

Solution Checklist

- State brute force first, then optimized approach.
- Confirm input constraints before coding.
- Dry run one normal case and one edge case.
- End with complexity and production tradeoffs.
- For design rounds, define requirements before drawing components.

PYQ-Style Attempt Protocol

- Step 1: identify topic and difficulty before solving.
- Step 2: write the first approach, even if it is brute force or long method.
- Step 3: optimize only after the basic idea is correct.
- Step 4: dry run one normal case and one edge case.
- Step 5: add answer, time, mistake type, and retest date to the log.

Pattern References

- Common OA patterns: hash map lookup, interval sorting, BFS/DFS, binary search on answer, DP state transition, stack/queue simulation.

Answer Review Table

| Mark | Meaning | Next Action |

| Solved | correct under timer | revise after 7 days |

| Guessed | answer came without method | solve two similar questions |

| Wrong | concept or execution gap | write mistake rule and reattempt tomorrow |

| Skipped | time or confidence issue | learn shortcut or move to lower difficulty |

Execution Dashboard

Metric	Target	Review Frequency
DSA Questions	10-14 per week	every Sunday
CS Core Notes	4 topics per week	twice a week
Mock Tests	1-2 per week	same day analysis
Interview Practice	5 answers per week	record and review

Weekly Review Ritual

- Pick the top 5 mistakes, rewrite the correct rule, and solve one similar question.
- Remove one weak resume line or prepare one stronger project answer.
- Decide the next week from evidence, not mood.

Final Self Audit

Score	Meaning	Action
0-40	reading stage	finish basics and easy questions
41-70	practice stage	increase timed mocks and mistake review
71-85	interview-ready	polish project, HR, and weak topics
86-100	selection-ready	maintain revision and avoid overloading new material

Source Note

- This is original Study Hub premium material for learning and practice. It is not copied from official or proprietary company papers.